Assignment 1 – Question 2: Process Management

1. Introduction

Process management is an important function of an operating system. It involves creating, executing, scheduling and terminating processes.

This assignment demonstrates process management using the C programming language on Parrot OS. The program creates a child process using the `fork()` system call, replaces the child process with another program using `execlp()`, and uses `waitpid()` to allow the parent process to wait for the child to finish.

2. Objectives

The objectives of this implementation are to:

- Create a child process using `fork()`.
- Identify parent and child processes using process IDs.
- Execute another program using `execlp()`.
- Allow the parent process to wait for the child process.
- Check the termination status of the child process.

3. Programming Language and Tools

Item	Description
Programming language	C
Operating system	Parrot OS (Linux)
Compiler	GCC
Editor	Kate
Header files	`stdio.h`, `unistd.h`, `sys/types.h`, `sys/wait.h`
System calls	`fork()`, `execlp()`, `waitpid()`

4. Implementation

4.1 Process Creation

The program uses the `fork()` system call to create a child process.

```
pid_t pid = fork();
```

The return value determines what happens next:

- A negative value indicates that process creation failed.
- A value of zero identifies the child process.

- A positive value identifies the parent process and contains the child's process ID.

4.2 Child Process Execution

The child process displays its process ID using `getpid()`.

It then executes another program using:

```
execlp("echo", "echo",
       "Child program replaced using execlp()",
       (char *)NULL);
```

The `execlp()` function replaces the current child process program with the `echo` command. If successful, the original child program does not continue executing.

If execution fails, the program displays an error using `perror()` and terminates the child using `_exit()`.

4.3 Parent Process and Synchronization

The parent process displays its own process ID and the child's process ID.

It then uses:

```
waitpid(pid, &status, 0);
```

This makes the parent wait for the specific child process to terminate.

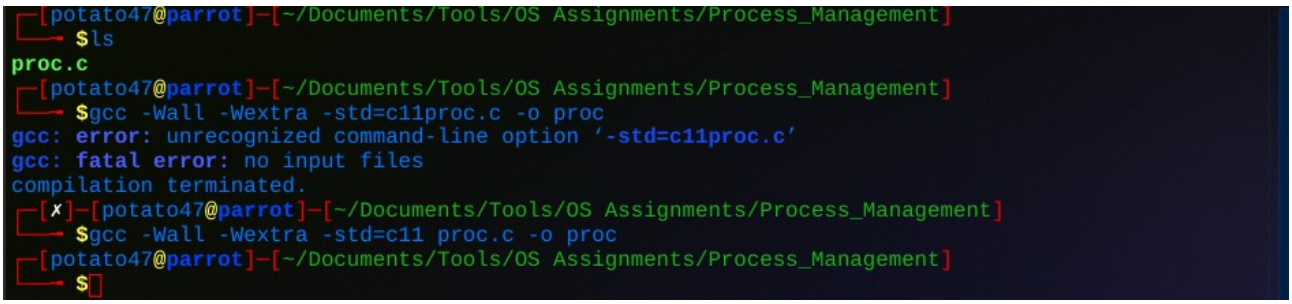
The program checks the child's termination status using:

- `WIFEXITED(status)` to determine whether the child terminated normally.
- `WEXITSTATUS(status)` to retrieve the child's exit status.

Finally, the parent displays a message indicating that execution has finished.

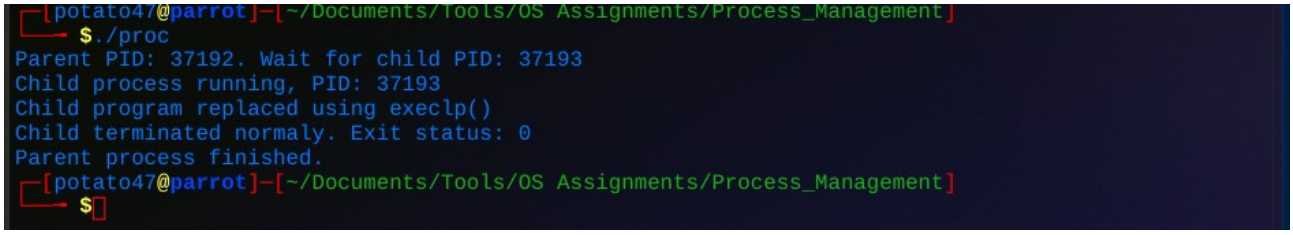
5. Compilation and Execution

The program was compiled using GCC with the following command: `gcc -Wall -Wextra -std=c11 proc.c -o proc`



6. Testing and Results

The compiled program is executed using ./proc .



The actual order of the parent and child messages may vary because the operating system scheduler determines when each process executes. The process IDs will also differ between executions.

7. Discussion

The implementation demonstrates important process management operations provided by the operating system.

The `fork()` system call creates a separate child process. Both the parent and child can then execute independently.

The `execlp()` function demonstrates program execution by replacing the child process's current program with the `echo` command. The child retains its process identity while executing the new program.

The `waitpid()` system call provides synchronization by preventing the parent from proceeding past the wait operation until the specified child terminates.

The use of `WIFEXITED()` and `WEXITSTATUS()` allows the parent to examine how the child process terminated.

One limitation of this implementation is that it demonstrates a single parent-child process relationship rather than managing multiple child processes or implementing a scheduling algorithm.

8. Conclusion

The assignment demonstrates basic process management using C on Parrot OS.

The program successfully illustrates process creation using `fork()`, program execution using `execlp()`, and process synchronization using `waitpid()`.

It also demonstrates how the parent process can retrieve and examine the termination status of its child process.

9. References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. *Operating System Concepts*. Wiley.
2. Linux Manual Pages: `fork(2)`, `exec(3)`, `getpid(2)` and `waitpid(2)`. https://man7.org/linux/man-pages/

3. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. *Operating Systems: Three Easy Pieces*. https://pages.cs.wisc.edu/~remzi/OSTEP/